A Correctness-First Introduction to Computer Science

Vaani Goenka
vaani.goenka_ug2024@ashoka.edu.in
Ashoka University
Sonipat, Haryana, India

Aalok Thakkar
aalok.thakkar@ashoka.edu.in
Ashoka University
Sonipat, Haryana, India

Abstract

Introductory computer science courses commonly use programming for instruction and assessment. This has several weaknesses because working code is not a reliable proxy for complete *understanding*. Elements of instruction – mathematics, a specific programming language, syntax, test-cases – all come together to conflate the boundaries and the specific purposes of these tools.

We propose a correctness-first pedagogical framework for introductory computer science that separating semantic reasoning from syntactic implementation. The framework structures problem solving into four epistemic stages: (1) blueprint specifications expressed as pre- and postconditions, (2) natural-language operational steps that make program behaviour explicit, (3) code implementation, and (4) proof of correctness. Central to this approach is a natural-language intermediate layer that enables students to reason about computation independently of programming syntax, making formal correctness reasoning accessible in early coursework.

We describe BOOP, a tool that implements this framework through lecture-integrated workflows and assignment support. BOOP is delivered via a web-based IDE and a VS Code extension, and provides automated checking of loop invariants and specifications. We report on a classroom deployment across two semesters (48 students). Qualitative analysis indicates earlier and more explicit engagement with correctness criteria, improved handling of edge cases, and increased willingness to reason beyond test inputs.

Our results suggest that correctness-first instruction is both pedagogically feasible in introductory CS courses and necessary for realigning assessment practices with foundational computational thinking goals in the presence of increasingly capable code-generation tools.

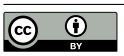
CCS Concepts

• **Social and professional topics → Computer science education**.

Keywords

Computational Thinking, Proof Writing, Formal Methods, Automated Grading

ACM Reference Format:
Vaani Goenka and Aalok Thakkar. 2026. A Correctness-First Introduction to Computer Science. In *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2026), July 10–15, 2026, Madrid, Spain.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3803400.3809367



ITiCSE 2026, Madrid, Spain

ACM ISBN 979-8-4007-2634-7/2026/07
https://doi.org/10.1145/3803400.3809367

1 The Limits of Programming

Programming occupies a central role in CS-1 coursework, typically administered through assignments, homework problems, and in-class code walkthroughs. This is a reasonable pedagogical choice — but it carries several underexamined challenges.

The most consequential of these is that working code is treated as an evidence of understanding. This proxy relationship emerges naturally: code is a concrete deliverable that can be graded automatically, making it an administratively convenient stand-in for the reasoning it is meant to represent. However, code is a weak proxy for the students' understanding. Instructors have consistently reported that students who produce working code cannot explain what it does or why it works [12, 13, 19].

This problem is compounded when instruction is organized around a specific programming language. Students trained in this way tend to acquire language-specific skills rather than transferable abstraction capabilities [2, 11, 22]. Programming assignments also impose a readiness burden that many CS-1 students have not yet met: without adequate prerequisite mental models, the cognitive overload of implementation in a specific language hinder rather than encourage the underlying concepts [3, 6, 17, 20].

The cumulative effect is that students learn to optimize for code that works rather than code that is correct [1]. This distinction is not cosmetic. Computer science is fundamentally a discipline of specification, reasoning, and formal justification — not empirical tinkering [5]. When students internalize working code as the goal, they are not merely developing bad habits; they are also forming a mistaken picture of what the discipline is [5, 21].

1.1 The Problem of Syntax-First Instruction

When courses prioritize direct instruction of language constructs and syntax, students systematically miss the necessity of abstraction. This encourages surface-level pattern matching over principled reasoning. This leads to a lack of *strategic knowledge*: the ability to recognize which abstraction applies to a novel problem [18]. Students become proficient at reproducing code patterns from examples but remain unable to derive solutions from first principles or transfer their knowledge to unfamiliar contexts [16]. Even more problematically, they develop erroneous mental models of computation based on inappropriate analogies to their previous academic experiences, particularly from mathematics [3, 6, 17, 20].

Pea identifies what he calls the "superbug": the persistent and pernicious belief that the computer "understands" what the student intends based on the logic of the problem domain [15]. Students approach programming as they would a mathematical proof, assuming the system will interpret intent, fill in missing steps, and

forgive imprecision. In practice, this belief manifests in debugging behaviour: compiler errors are treated as syntactic obstacles to bypass rather than as evidence that the program fails to specify valid computational behaviour. Students iteratively patch symptoms until test cases pass, rather than diagnosing and correcting specification violations.

True correctness, however, should derive from sound mathematical reasoning applied systematically, not emerge accidentally from iterative empirical refinement.

1.2 Foundational Disconnection

Our experience, as well as empirical research [7], demonstrates that students often fail to see the connections between mathematical foundations and computer science. This stems from two related factors: First, many students experience mathematics primarily as computation, and when they encounter CS, they apply this mindset to code, seeking algorithms to execute rather than specifications to satisfy. Second, students enter computing courses from diverse backgrounds with varying mathematical preparation. A pedagogical approach grounded in first-principle reasoning and constructivist learning theory provides the necessary scaffolding for this heterogeneous population. Without such scaffolding, students may learn a specific programming language rather than the foundations of computational thinking that transcend any particular notation.

To address these challenges, we propose a correctness-first lens for introductory computer science that directly introduces a stronger tool for learners to engage in computational thinking. In this approach, learning to program emerges as a consequence of learning to reason about computation. This perspective builds on established foundations. Edsger Dijkstra introduced the *weakest precondition* calculus, and showed that programs can be systematically derived from specifications [4]. David Gries extended this view, emphasizing that students who learn to assign meaning through formal reasoning develop deeper cognitive models than those trained primarily in syntax [9]. He argued that the true meaning of a program resides in the programmer's internal theory, with code serving as a secondary artifact, and that instruction should focus on assigning programs to meanings (specifications) rather than meanings to programs [14].

1.3 Research Questions

This work is guided by the following research questions:

(1) **RQ1:** How does a correctness-first instructional framework affect novice students' ability to articulate formal specifications and reason about edge cases in introductory programming tasks?
(2) **RQ2:** Can a structured separation between specification, operational reasoning, implementation, and proof make formal reasoning accessible in introductory CS courses without prior exposure to formal methods?
(3) **RQ3:** What pedagogical benefits and challenges arise when integrating correctness-first workflows and tool support into existing introductory computer science curricula?

2 A Correctness-First Pedagogical Framework

We propose a correctness-first pedagogical framework that structures problem solving around explicit reasoning *prior* to implementation. This approach fundamentally redefines programming: it is not a process of empirical discovery through execution, but the act of encoding a solution whose correctness has already been argued.

Designed to address persistent challenges such as overemphasis on syntax and the disconnection between code and formal meaning, the framework is guided by four pedagogical principles:

(1) **Correctness precedes code.** Students must articulate what it means for a program to be correct before writing implementation. This reverses the dominant output-driven assessment model, treating correctness as a semantic property to be specified and justified rather than a post-hoc property inferred from a test harness.
(2) **Separation of epistemic tasks.** Specification, reasoning, implementation, and proof are treated as distinct stages. This separation reduces cognitive overload, prevents syntax from obscuring conceptual reasoning, and allows instructors to diagnose the precise source of student difficulty (e.g., distinguishing a logic error from a syntax error).
(3) **Natural-language reasoning as a bridge.** Unlike traditional formal methods that rely on early symbolic logic, we utilize a natural-language operational stage. This bridges informal reasoning and formal code, allowing novices to articulate step-by-step transformations without the confounding influence of syntactic detail.
(4) **Derivation over discovery.** Programs are derived from specifications through invariants. Loops and recursion are treated as mechanisms for preserving explicitly stated properties, not as exploratory constructs whose behaviour is discovered through trial-and-error debugging.

To operationalize these principles, the framework organizes every programming task into four explicitly delineated stages: *Blueprint Specification*, *Operational Reasoning*, *Implementation*, and *Proof of Correctness* [8]. Each stage produces a concrete artifact with a distinct epistemic role, ensuring that code is treated as the final representation of a verified thought process rather than the starting point of inquiry.

2.1 The Four-Stage Workflow

Each problem (in lectures, discussion sessions, and assignments) is structured under the following framework:

(1) **Blueprint Specification.** In the Blueprint stage, students formalize what it means for a solution to be correct. This is expressed as a function contract consisting of explicit *preconditions* and *postconditions*, written independently of any implementation strategy.
This stage targets a common manifestation of test-case myopia: underspecified solutions that pass observed examples but fail in untested cases. By requiring explicit contracts, students are encouraged to reason about minimal models that satisfy their specifications and to identify missing constraints early. Our implementation supports automated checking of blueprint specifications using property-based testing.

(2) **Operational Reasoning.** In the Operational Reasoning stage, students describe the conceptual steps that transform inputs satisfying the preconditions into outputs satisfying the postconditions in natural language. This is inspired by Knuth's literate programming [10], and helps externalizes students' internal theories of computation, making reasoning inspectable, debuggable, and assessable.
This is also the stage at which invariants, progress measures, or recursive decompositions naturally emerge, providing a principled bridge between specification and implementation.
(3) **Implementation.** Only after specification and operational reasoning are complete do students translate their solution into code. Because the structure of the solution has already been justified, implementation becomes an exercise in faithful representation rather than exploratory problem solving. Our current tooling supports Python and OCaml. The parser can prohibit constructs that obscure reasoning (see Table 1), with constraints configurable by the instructor.
(4) **Proof of Correctness.** In the final stage, students justify that their implementation satisfies the original function contract. This involves arguing that the operational reasoning is correctly encoded in the program and that key properties, such as invariants, are preserved.
Correctness is treated as independent of test cases. While testing may still be used for feedback, the proof stage reinforces the idea that correctness is a semantic property rather than an empirical one. Partial automation is provided through property-based checking of invariants.

3 Lecture Integration and Curriculum Design

Correctness-first approach imposes epistemic expectations that must be explicitly supported in lectures. When lectures remain syntax-first or example-driven, students experience correctness requirements as artificial or punitive rather than as natural consequences of disciplined reasoning. In response, we redesigned lecture sequencing, in-class activities, and worked examples to mirror the structure of the correctness-first framework and to model the style of program derivation advocated by David Gries [9].

Rather than presenting programming as the production of executable artifacts, lectures emphasize the construction and justification of meaning: how specifications constrain behavior, how invariants preserve meaning across steps, and how implementations arise as representations of prior reasoning. This alignment is essential for shifting students away from test-case–driven heuristics toward semantic reasoning as the primary mode of understanding.

3.1 Reordering Concept Introduction

A central change concerns the order in which core concepts are introduced. Traditional introductory courses often begin with control flow and syntax, deferring questions of correctness until after students have already developed trial-and-error habits. To counter this, lectures are organized around the following progression:

(1) Formal problem specifications
(2) Semantic reasoning about state transformations
(3) Invariants and termination arguments
(4) Syntactic realization in code

This ordering ensures that students encounter reasoning principles in lectures before being expected to apply them in assignments. Control structures such as loops and recursion are introduced not as language features, but as mechanisms for preserving stated properties under repeated state transitions.

3.2 Demonstrative Example: Sorting a List

We illustrate this shift using a canonical example: sorting a list. When asked to specify what it means for a list to be sorted, most students initially provide only an ordering condition, such as:

$$\forall i < j,\ a_i \leq a_j.$$

In lectures, we make explicit that this condition alone is insufficient, and bring up counter-examples. Students try to refine their specifications (sometimes incorrectly by adding that every element in the input list must occur in the output list), and finally arrive at the correct answer. This observation typically surprises students and exposes a common manifestation of lacking a mental model of computation: they implicitly assume untested properties without articulating them.

We therefore guide students to refine the specification by adding a permutation condition:

output is a permutation of the input.

This refinement is performed live in lecture, with students asked to propose candidate specifications and then challenged to construct counterexamples. The exercise serves multiple purposes: it demonstrates the necessity of precise specifications, reveals hidden assumptions, and shows how correctness criteria arise from reasoning rather than from observed behaviour on examples. Importantly, this discussion precedes any implementation of a sorting algorithm. Such an activity is essential before introducing the algorithm for any computational problem.

Having established the complete specification for sorting—both the ordering and permutation conditions—we extend the pedagogical value of this example by implementing multiple sorting algorithms. This approach reinforces a crucial principle: a single specification can be satisfied by different implementations, each relying on distinct loop invariants and structural insights. We typically present three algorithms in sequence: selection sort, insertion sort, and merge sort. Invariants are not discovered mechanically from specifications, but rather emerge from algorithmic insights. Students observe that while all three implementations satisfy identical preconditions and postconditions, the reasoning required to verify each is qualitatively different. The choice of algorithm, and consequently the choice of invariant reflects different trade-offs in time complexity, space usage, and conceptual simplicity.

By presenting these alternatives, we also combat the assumption that there exists a single "correct" way to solve a problem. Students learn that specifications describe *what* must be achieved, while invariants and implementations describe *how* it can be achieved—and that the latter admits creative diversity within the constraints of the former.

3.3 Backward Reasoning and Gries-Style Derivation

We then lectures incorporate backward reasoning exercises inspired by weakest-precondition reasoning [7]. Given a desired postcondition and a fragment of a program, students are asked to infer the weakest precondition under which the fragment is correct. This is achieved by introducing the concept of *wp-calculus* without introducing the formal (and overwhelming) terminology.

Students are provided an assignment statement and a postcondition and asked:

> What must be true *before* this step in order for the postcondition to hold afterward?

These exercises begin with simple problems (for example avoiding division-by-zero errors) and build up to more sophisticated ideas such as list operations, and manipulation of data-structures. In lecture, the backward reasoning is treated as a way of making semantic dependencies explicit. This directly supports both the Blueprint and Proof stages of the framework.

3.4 Invariants as Central Lecture Objects

To prevent students from treating loops as 'trial-and-error' constructs, we introduce invariants not as new formalisms, but as familiar inductive hypotheses from recursion, and then revisited during programs involving iteration. Rather than treating invariants as an advanced or auxiliary concept, we structure the course so that invariants emerge naturally from reasoning patterns students already understand.

The course begins exclusively with purely functional and recursive programs. In this setting, program state is explicit: it is fully represented by the arguments of a function. Correctness proofs are carried out using structural induction (and induction is taught as a part of the course). This allows early lectures to focus on the correspondence between recursive definitions and inductive reasoning without introducing additional semantic complexity.

We devote approximately one third of the course to writing and reasoning about recursive programs over simple data types such as integers, booleans, and lists. During this phase, students learn to articulate preconditions and postconditions and to structure proofs that mirror the recursive structure of the program. Correctness is presented as a property derived from the definition, not as an empirical observation.

We then transition to tail recursion and explicitly demonstrate how structurally recursive functions can be transformed into tail-recursive ones by making intermediate state explicit through accumulator parameters. This transformation is performed in lecture, with attention to how the inductive hypothesis must be strengthened to account for the accumulator. Students observe that what was previously implicit in the call stack must now be expressed as an invariant relating the accumulator to the original input.

Finally, we show that imperative loops are semantically equivalent to tail-recursive functions, differing only in how state is represented. In loops, the state becomes implicit in mutable variables rather than explicit in function arguments. At this point, invariants are introduced not as a new concept, but as a reinterpretation of the inductive hypothesis for programs with implicit state. Students are encouraged to view loop invariants as the statements that would have appeared as inductive hypotheses had the program been written recursively.

This progression significantly lowers the conceptual barrier to invariants. Rather than learning invariants as a formal technique attached to loops, students recognize them as a familiar reasoning tool applied in a new representational context. Additionally, this challenges the students who are used to imperative programming from high school to rethink of programming as a reasoning exercise.

3.5 Termination and Ranking Functions

Termination is treated as a first-class concept, explicitly separated from functional correctness. Lectures introduce ranking functions as mappings from program states to a well-founded set (typically $\mathbb{N}$) and emphasize their two defining properties:

(1) **Non-negativity:** $L \implies f \geq 0$
(2) **Strict decrease:** $f_{\text{before}} > f_{\text{after}}$

This treatment builds directly on students' prior experience with recursive definitions, where termination is often implicit. By making termination arguments explicit, students learn to distinguish between two logically independent questions: whether a program halts, and whether it produces a correct result upon halting. Students learn that termination is necessary but insufficient, and that correctness requires additional guarantees.

4 BOOP: Tool Support for Assignments

Correctness-first instruction cannot be sustained through lectures alone. Assessment structure and tooling must reinforce the same epistemic commitments. We therefore pair our lectures with targeted tool support. The goal of this infrastructure is to provide timely, conceptually aligned feedback that reinforces disciplined reasoning without overwhelming novice learners. We developed a lightweight tool suite, *BOOP*, [1] that operationalizes the correctness-first workflow while remaining flexible enough for incremental adoption. The tooling supports students in articulating specifications, maintaining invariants, and justifying correctness, without requiring exposure to full formal verification systems.

Architecture and Deployment. The BOOP tool suite is delivered as both a VS Code extension and a Web-based IDE. From the student's perspective, the two environments provide equivalent functionality and share a common back-end for parsing, analysis, and feedback generation. The Web IDE additionally supports centralized logging, enabling instructors to observe patterns in student reasoning and tool usage across assignments.

Language Support. The implementation supports OCaml and Python. OCaml reflects its use in our introductory curriculum and its suitability for reasoning about recursion and immutability. Python support was added to evaluate the framework's applicability beyond functional programming and to lower barriers to adoption in more conventional settings.

While the tool is historically named BOOP, which stands for *Blueprint-Operations-OCaml-Proof*, the pedagogical framework itself is language-agnostic. Language-specific frontends allow the

[1] The tool is available at: https://use-boop-production.up.railway.app

same reasoning checks and feedback mechanisms to be applied across languages.

Preprocessing. An obstacle in correctness-first instruction is that certain implementation choices obscure reasoning rather than support it. BOOP performs a configurable preprocessing step that flags constructs likely to undermine reasoning.

Table 1: Preprocessing Flags by Language

Preprocessing Flag	Rationale
OCaml	
Use of built-in library functions	Prohibited when the instructional goal is to derive the underlying algorithm explicitly (e.g., `List.rev`)
Deeply nested local function definitions	Avoided to prevent fragmented control flow and hinder stepwise reasoning; flattening definitions encourages program structure to correspond more directly to operational reasoning
Anonymous, shadowed, or reused variable names	Prohibited to prevent obscuring data flow and weakening the traceability of reasoning across specification, invariant, and implementation stages
Implicit or omitted type annotations	Students are required to provide explicit types for function signatures and key intermediate values
Python	
Mandatory type annotations	Required for function signatures to support reasoning about correctness and enable automated checking, despite not being enforced by the language runtime
Restricted use of library abstractions	Prohibited when these abstractions (e.g., list comprehensions, higher-order functions, or built-in aggregations) would allow students to skip explicit reasoning required by the assignment
Prohibition of implicit control flow	Early `return` statements scattered throughout a function or reliance on exceptions for normal control flow are prohibited, as they can obscure invariants and termination arguments
Explicit loop structure requirements	Encourages simple, traceable loops whose guards and updates correspond directly to stated invariants and ranking functions

These restrictions are not intended to define best practices for professional programming, but to scaffold the development of reasoning skills in novice learners. Instructors can selectively enable, relax, or disable individual checks based on assignment goals, course progression, and student readiness.

Invariant Type	Tool Behavior
Sound and complete	Verification succeeds
Incorrect	Counterexample likely generated
Sound but incomplete	Counterexample illustrating gap

Table 2: Tool behaviour for different invariants

Automated Grading of Blueprint. BOOP automates grading of student solutions by checking blueprint properties using *property-based testing*. For this, BOOP generates a verification harness around the student implementation: the pre-condition and postcondition properties are asserted at appropriate places in the functions, and inputs are systematically generated using *Hypothesis* (Python) or *QCheck* (OCaml) tools. During execution, any violation of the specified properties produces a counterexample, which is returned as actionable feedback. This approach allows automated, fine-grained assessment of correctness and adherence to design constraints, without relying on a fixed set of test cases.

Invariant-Oriented Feedback. To reinforce invariant-based reasoning without demanding fully mechanized proofs, the tool provides lightweight automated feedback using property-based testing. Loops annotated with preconditions, postconditions, and invariants are instrumented and checked using **QCheck** for OCaml and **Hypothesis** for Python.

Two complementary checks are performed:

(1) **Invariant Soundness.** To check if the invariants are preserved, the tool constructs a verification harness in which the precondition is assumed, invariants are asserted before loop entry, and reasserted after each iteration. Counterexamples generated during testing indicate violations of invariant preservation and are presented to students as concrete failing states.

(2) **Invariant Completeness.** To assess whether invariants are sufficient to imply the postcondition upon termination, the tool checks the satisfiability of:

$$I \wedge \neg L \wedge \neg P.$$

If satisfiable, this indicates a terminating state consistent with the invariant that violates the postcondition, revealing an incomplete correctness argument.

Table 2 summarizes the resulting feedback.

From a curricular perspective, the framework integrates naturally into introductory courses without requiring a separate formal methods module. By explicitly confronting weaker pedagogical practices related to programming and foregrounding correctness-first reasoning, it aligns instruction with foundational computational thinking goals while remaining accessible to students with diverse mathematical backgrounds.

5 Discussion and Conclusion

This work addresses a persistent but underexamined problem in introductory computer science education: the tendency for students to treat passing test cases as a proxy for program correctness. We proposed a correctness-first pedagogical framework that

places specification, reasoning, and justification before implementation, and described how this framework can be supported through aligned lectures and lightweight tooling. The small-scale classroom deployment reported here is intended as illustrative evidence of feasibility, not as empirical validation of the framework's efficacy.

5.1 Addressing the Research Questions

We deployed our framework and tooling in [CS-1102] Introduction to Computer Science at Ashoka University with 45 students across two iterations.

RQ1: Articulation of specifications and reasoning about edge cases. We observed, through instructor notes and submitted assignment artifacts (specifications, operational descriptions, and proofs), that students more frequently stated complete correctness conditions, identified missing constraints, and recognized when solutions that passed all provided tests were nonetheless incorrect, compared to previous iterations of the course. Separating specification from implementation was practically significant: even when implementations were incomplete, many students demonstrated sound reasoning in their specifications and operational descriptions. This suggests that conceptual understanding and syntactic proficiency can develop somewhat independently, consistent with the Gries-style view that a program's meaning resides in the programmer's internal theory, with code serving as its representation.

RQ2: Accessibility of formal reasoning without prior exposure. Our experience suggests that formal reasoning can be made accessible in introductory courses through a structured separation of tasks. The natural-language operational reasoning stage was central to this: it allowed students to externalize and refine their thinking in an inspectable form that could be gradually made more precise. Instructor observations across both iterations noted that students engaged with invariants and correctness arguments earlier than in previous cohorts, and many did not experience these activities as "formal methods" in the traditional sense—a framing we deliberately avoided.

RQ3: Pedagogical benefits and challenges of integration. Integrating correctness-first workflows yielded several observable benefits alongside predictable challenges. The decomposition into specification, reasoning, implementation, and proof enabled finer-grained assessment: instructors could distinguish conceptual gaps from coding difficulties, and students with syntactically correct code but weak specifications were prompted to revisit their understanding rather than being rewarded for surface correctness.

The primary challenge, evident in submitted artifacts, was student resistance to treating specification as a task independent of implementation. A recurring pattern was students deriving correctness conditions *from* their code rather than prior to it—restating what the implementation happened to do rather than characterizing what it should do. Addressing this required sustained instructional reinforcement; it did not resolve on its own once the framework was introduced. This underscores a broader point: correctness-first workflows cannot succeed when instruction remains syntax-first, and the effort required to align lectures, assignments, and assessment around the same epistemological commitments should not be underestimated.

Tool support proved valuable but secondary. The BOOP tool reinforced classroom norms and surfaced counterexamples that challenged premature closure on implementations, but its effectiveness stemmed from integration into a coherent pedagogical philosophy rather than from the automation itself.

5.2 Implications and Scope

There is something telling about the fact that "correctness-first" requires a name at all. In most disciplines, the idea that a design must be correct—not merely untested to failure—is not a pedagogical stance but a baseline. A civil engineer does not certify a structure because it survived a handful of load tests; a pharmacist does not approve a compound because spot checks returned no immediate harm. That computer science education has drifted far enough from this baseline that correctness must be explicitly foregrounded, advocated for, and given a label is itself a diagnosis. The prevalence of test-driven cultures, autograders, and now AI code generation has made this drift more visible: when a student can obtain passing outputs without constructing any theory of why their program is correct, the absence of that theory goes undetected until it matters. This disruption is, in our view, not fundamentally about automation but about epistemology. AI-generated code does not create the problem of equating test-passing with correctness; it merely exposes it. A correctness-first approach does not introduce a new standard. It attempts to recover one that should never have been abandoned, and to re-center instruction around specification and justification in a moment when doing so is newly urgent.

What is at stake is not only student preparation for advanced coursework, but the disciplinary identity of computer science itself. Programming has always admitted a wide range of practitioners, but computer science as a field is grounded in the idea that computational behaviour can be reasoned about, not merely observed. When introductory courses implicitly teach that observation is enough, they do not just produce graduates who write fragile code; they produce graduates who have absorbed a fundamentally impoverished model of what it means to understand a program. Correctness-first pedagogy is, in this sense, not a curricular preference but a question of intellectual honesty about what the discipline is.

The consistency of observed patterns across instructor notes and assignment artifacts suggests the approach is worth sustained and more rigorous investigation. Future work should examine larger deployments, employ more systematic evaluation designs, including direct analysis of student-produced artifacts against defined rubrics and, where feasible, comparison with programming-first cohorts, and attend to longitudinal retention of reasoning skills. More granular accounts of where the framework succeeded and where students continued to struggle will be essential to building an honest evidence base.

Acknowledgement. We gratefully acknowledge the support of the Mphasis Foundation for funding this research.

References

[1] Marzieh Ahmadzadeh, Dave Elliman, and Colin Higgins. 2005. An Analysis of Patterns of Debugging Among Novice Computer Science Students. In *Proceedings of the 10th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '05)*. ACM, 84–88. doi:10.1145/1067445.1067472

[2] Michal Armoni. 2013. On Teaching Abstraction in Computer Science to Novices. *Journal of Computers in Mathematics and Science Teaching* 32, 3 (2013), 265–284.

[3] Mordechai Ben-Ari. 2001. Constructivism in Computer Science Education. *Journal of Computers in Mathematics and Science Teaching* 20, 1 (2001), 45–73.

[4] Edsger W. Dijkstra. 1976. *A Discipline of Programming*. Prentice-Hall.

[5] Edsger W. Dijkstra. 1989. On the Cruelty of Really Teaching Computing Science. *Commun. ACM* 32, 12 (1989), 1398–1404. doi:10.1145/74588.74589

[6] Allan E. Fleury. 1991. Parameter Passing: The Rules the Students Construct. *ACM SIGCSE Bulletin* 23, 1 (1991), 283–286.

[7] Timothy S. Gegg-Harrison, Gary R. Bunce, Rebecca D. Ganetzky, Christina M. Olson, and Joshua D. Wilson. 2003. Studying Program Correctness by Constructing Contracts. In *Proceedings of the 8th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '03)*. ACM, 129–133.

[8] Vaani Goenka and Aalok Thakkar. 2026. BOOP: Write Right Code. In *Computing Education Research (Communications in Computer and Information Science, Vol. 2739)*, P. Prasad, A. Raman, and B. Shah (Eds.). Springer, Cham. doi:10.1007/978-3-032-14583-3_7

[9] David Gries. 1981. *The Science of Programming*. Springer-Verlag.

[10] Donald E. Knuth. 1992. Literate Programming. In *Literate Programming*. CSLI. Originally published in 1984.

[11] Jeff Kramer. 2007. Is Abstraction the Key to Computing? *Commun. ACM* 50, 4 (2007), 36–42.

[12] Teemu Lehtinen, Aleksi Lukkarinen, and Lassi Haaranen. 2021. Students Struggle to Explain Their Own Program Code. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1* (Virtual Event, Germany) *(ITiCSE '21)*. Association for Computing Machinery, New York, NY, USA, 206–212. doi:10.1145/3430665.3456322

[13] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Mary Lindholm, Robert McCartney, Jan Ericsson Moström, Kate Sanders, Otto Seppälä, Beth Simon, and Lynda Thomas. 2004. A Multi-national Study of Reading and Tracing Skills in Novice Programmers. *ACM SIGCSE Bulletin* 36, 4 (2004), 119–150.

[14] Peter Naur. 1985. Programming as Theory Building. *Microprocessing and Microprogramming* 15, 5 (1985), 253–261.

[15] Roy D. Pea. 1986. Language-Independent Conceptual "Bugs" in Novice Programming. *Journal of Educational Computing Research* 2, 1 (1986), 25–36.

[16] D. N. Perkins, C. Hancock, R. Hobbs, F. Martin, and R. Simmons. 1986. Conditions of Learning in Novice Programming. *Journal of Educational Computing Research* 2, 1 (1986), 37–55.

[17] Anthony Robins, J. Rountree, and N. Rountree. 2003. Learning and Teaching Programming: A Review and Discussion. *Computer Science Education* 13, 2 (2003), 137–172.

[18] Marco Sbaraglia, Michael Lodi, and Simone Martini. 2021. A Necessity-Driven Ride on the Abstraction Rollercoaster of CS1 Programming. *Informatics in Education* 20, 4 (2021), 641–682. doi:10.15388/infedu.2021.28

[19] Simon and Susan Snowdon. 2011. Explaining program code: giving students the answer helps - but only just. In *Proceedings of the Seventh International Workshop on Computing Education Research* (Providence, Rhode Island, USA) *(ICER '11)*. Association for Computing Machinery, New York, NY, USA, 93–100. doi:10.1145/2016911.2016931

[20] Juha Sorva. 2013. Notional Machines and Introductory Programming Education. *ACM Transactions on Computing Education (TOCE)* 13, 2 (2013), 1–31.

[21] Jeannette M. Wing. 2006. Computational Thinking. *Commun. ACM* 49, 3 (2006), 33–35. doi:10.1145/1118178.1118215

[22] Leon E. Winslow. 1996. Programming Pedagogy: A Psychological Overview. *ACM SIGCSE Bulletin* 28, 3 (1996), 17–22.